



UNIVERSITY OF
CAMBRIDGE

Research Informatics Training

Data analysis in R and Python

Cambridge Centre for Research Informatics Training

Housekeeping (in-person)



- **Introductions**

Trainers & participants



- **Attendance** sheet



- **Facilities**

Fire assembly, toilets, refreshments, WiFi



- Questions & **participation**



Housekeeping (online)



- **Introductions**

Trainers & participants



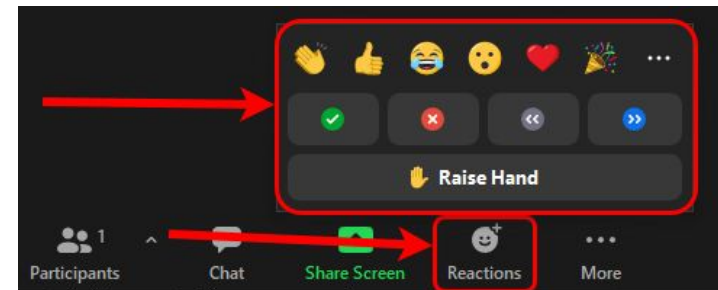
- **Attendance**

Zoom display name



- **Questions & participation**

General Zoom “etiquette”



Course aims

- Getting familiar with the programming language
- Confidence in exploratory data analysis
- Create plots to visualise data

By the end of this course you'll be able to read in, analyse, interpret and visualise data.

Plan for the course

Lectures	Short (10 - 15 minutes) Background/key concepts	
Practicals	Implementation in R/Python Support from trainers	
Live demo	Demonstration of software, examples	
Exercises	Exercises in the practicals	

Course layout: 4 half-day sessions

- DA1: Getting started (software, data types)
- DA2: Data & plotting (working with tabular data & plotting)
- DA3: Manipulating data (filtering, creating, grouping)
- DA4: Organise & combine (reshaping, combining data)



UNIVERSITY OF
CAMBRIDGE

Research Informatics Training

DA1: Getting started

Cambridge Centre for Research Informatics Training

Plan for the session

Session 1: **Getting started**

1. Coding as a tool in research
2. Core coding skills
3. Data types and structures

1. Coding as a tool in research

Setting the scene

Why we use programming in research

Programming encourages you to be:

1. Organised



2. Precise and consistent



3. Transparent & reproducible



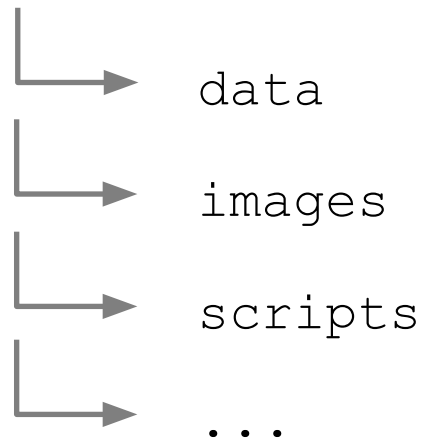
4. Collaborative



The working directory

- Keeps your analysis together & makes it easier to share.
- Starting point for the computer.

data-analysis



Starting point: absolute or relative

`data-analysis` with a data file in: `data/surveys.csv`

Absolute path:

`C:/Users/Jeff/Documents/data-analysis/data/surveys.csv`

The diagram shows the absolute path 'C:/Users/Jeff/Documents/data-analysis/data/surveys.csv' with three blue brackets underneath it. The first bracket is under 'C:/Users/Jeff/Documents', the second is under 'data-analysis', and the third is under 'data/surveys.csv'.

Computer-specific

Project folder

Path to file

Relative path:

`data/surveys.csv` (computer starts here: `data-analysis`)

2. Core coding skills

Learning to code

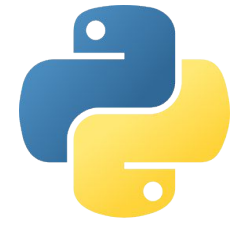
Language & interface

R



the language

Python



RStudio



the interface

JupyterLab



Running code

Console/terminal



- Runs code directly
- Interactive
- No record
- Tricky to trace

Script/Notebook



- Basically, a text file
- Needs to be told to run
- Records actions
- Transparent

Functions

- Functions perform a series of operations
- Automate more complex commands
- Often take *arguments* that allow control over how it behaves
- Many are predefined, but you can write your own

# example	# value
<code>abs(-12)</code>	12
<code>round(15.821)</code>	16
<code>round(15.821, 2)</code>	15.82
<code>round(15.821, digits = 2)</code>	15.82

Adding functionality

- Built-in functionality (base R, Python)
- Extra functionality can be added using packages/libraries
- Install once, load every time

Examples:

Usage	R	Python
Data manipulation & analysis	readr, dplyr (tidyverse)	pandas
Plotting	ggplot2	plotnine; matplotlib; seaborn
Numerical computing	base R	NumPy

Remember syntax or not?



ChatGPT 5 ▾

how do I round a number to 2 digits in R?

In R, you can round a number to 2 decimal places using the `round()` function.

R

Copy code

```
x <- 3.14159
rounded <- round(x, 2)
print(rounded)
# [1] 3.14
```

- The first argument is the number (or vector).
- The second argument (`digits`) specifies how many decimal places to keep.

Remember syntax or not?



ChatGPT 5 ▾

how do I round a number to 2 digits in python

In Python, you can round a number to 2 decimal places using the built-in `round()` function:

python

Copy code

```
num = 3.14159
rounded = round(num, 2)
print(rounded) # 3.14
```



- **Live demo** software, setting up a working directory, create a script, running code
- **Practical** work through Chapter 4 ([Intro to software](#))
- **Exercise** creating a working directory, creating a script/Notebook, running code, functions



3. Data types & structures

Different data for different purposes

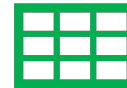
Why we use programming in research

Programming encourages you to be:

1. Organised



2. Precise and consistent



3. Transparent



4. Collaborative





Quantitative data

- Discrete



- Continuous



Qualitative data

- Categorical



- Ordinal



Numbers, text and logical

Important across all programming languages is how they view the data.

1. **Numerical** (integers or decimals, e.g. 9 or 12.82)
2. **Text** (character, e.g. "control")
3. **Logical** (boolean, e.g. TRUE or FALSE)
4. **Missing data** not the same as zero! (NA or NaN)



Data structures

Can be simple or complex:

- **Single objects**

e.g. an object called *temperature* containing the value 42.

Data structures

Object are there to be *used*. Most are not static and can be updated/changed:

For example:

# example	# value
temperature	42
temperature - 10	32
temperature * 2	84

Data structures

Can be simple or complex:

- **Collections of data**

e.g. *temps* containing the values 42, 12, 19, 23.

e.g. *weather* containing the values "sun", "rain", "cloud"

Data structures: collections



Collection / data	R	Python
Same type 1D	vector	NumPy array
Same type 2D	matrix / array	NumPy array
General container (mixed)	list	list / tuple
Tabular data (mixed)	data.frame / tibble	pandas DataFrame

Data subsetting

Subsetting data: selecting only part of the data.

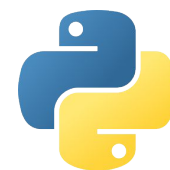
Each value has an index. Indexing style is language-dependent.
We can subset based on index or label.

```
temperature <- c(22, 22, 31, 42, 16)
```

```
temperature[2:3] gives 22, 31
```

```
temperature = [22, 22, 31, 42, 16]
```

```
temperature[2:3] gives 31
```



Data types & structures: practical & exercises



- **Live demo** data types, subsetting
- **Practical** work through Chapter 5
(Data types & structures)
- **Exercises** create & use objects,
updating objects



DA1: Summary

- Programming helps you to be **organised, consistent, transparent** and **collaborative**
- The **working directory** acts as a starting point

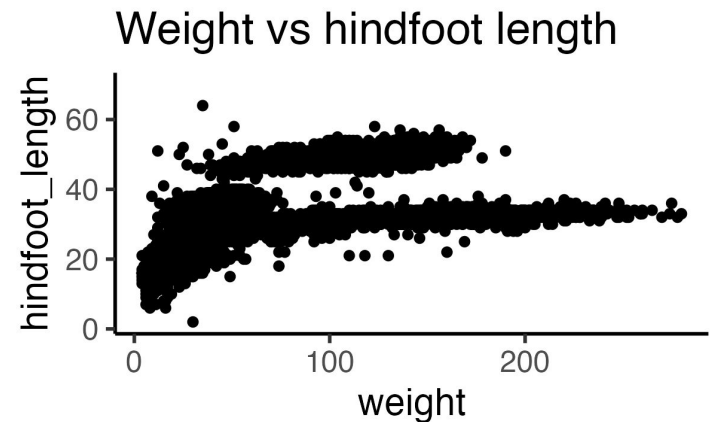
DA1: Summary

- Programming helps you to be **organised, consistent, transparent** and **collaborative**
- The **working directory** acts as a starting point
- Programming recognises different data types, such as **numbers, text, logical** and **missing values**
- These data can be stored in **objects**, which can contain one or more values - these can be static or dynamic

Coming up next...

DA2: Data & plotting

- Using tabular data (rows and columns)
- Learn how to visualise data





UNIVERSITY OF
CAMBRIDGE

Research Informatics Training

DA1: Getting started (end)

Cambridge Centre for Research Informatics Training



UNIVERSITY OF
CAMBRIDGE

Research Informatics Training

DA2: Plotting tabular data

Cambridge Centre for Research Informatics Training

Plan for the session

Session 2: **Plotting tabular data**

1. Working with tabular data
2. Plotting data

1. Working with tabular data

Why formats matter

Tabular data

- Great format for any programming language

record_id	hindfoot_length	weight	genus	taxa
182	32		Neotoma	Rodent
272	31		Neotoma	Rodent
652		182	Callipepla	Bird
854	28	47	Pipilo	Bird
977	107	1263	Sylvilagus	Rabbit

Tabular data

- Each variable (thing you measure) has its own column.

variables (columns)



record_id	hindfoot_length	weight	genus	taxa
182	32		Neotoma	Rodent
272	31		Neotoma	Rodent
652		182	Callipepla	Bird
854	28	47	Pipilo	Bird
977	107	1263	Sylvilagus	Rabbit

Tabular data

- Each observation has its own row.

observations (rows) ↓

record_id	hindfoot_length	weight	genus	taxa
182	32		Neotoma	Rodent
272	31		Neotoma	Rodent
652		182	Callipepla	Bird
854	28	47	Pipilo	Bird
977	107	1263	Sylvilagus	Rabbit

Tabular data

- Note: **missing data**

record_id	hindfoot_length	weight	genus	taxa
182	32		Neotoma	Rodent
272	31		Neotoma	Rodent
652		182	Callipepla	Bird
854	28	47	Pipilo	Bird
977	107	1263	Sylvilagus	Rabbit

Tabular data: quality control checks

- Check for missing data
- Check number of columns and rows
- Data types of each column
- Check/count unique values
- Summary statistics

Tabular data

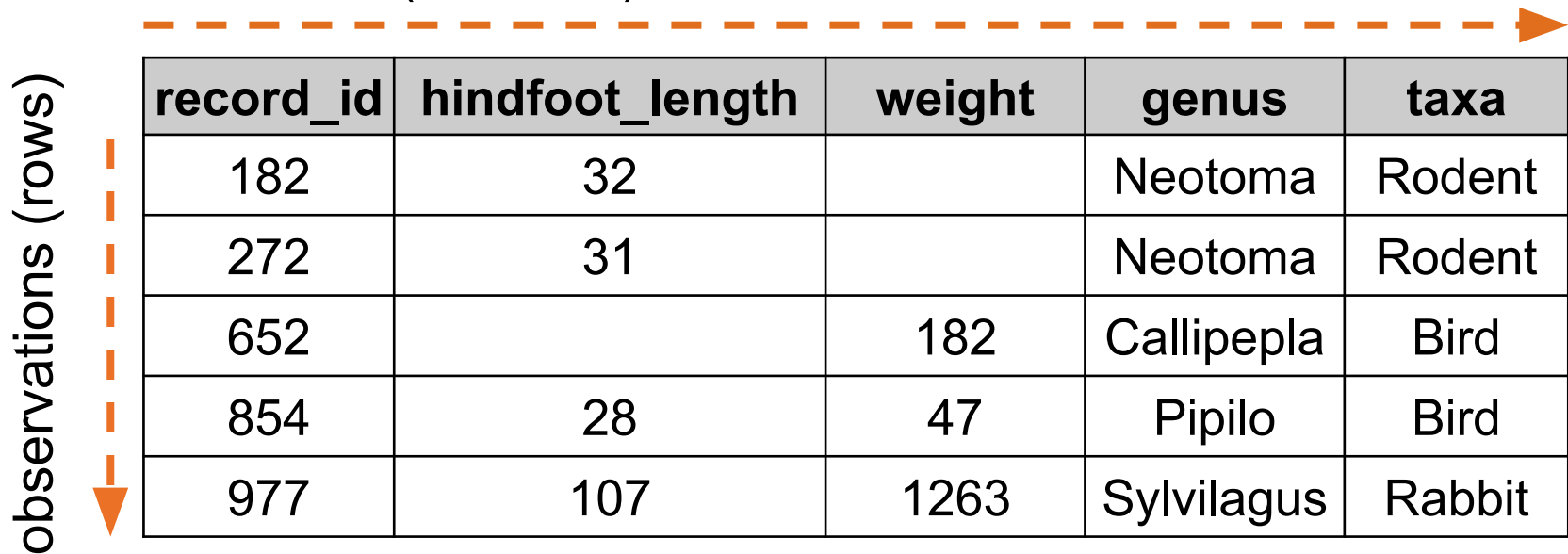
- Note: each observation is **uniquely identifiable**.

record_id	hindfoot_length	weight	genus	taxa
182	32		Neotoma	Rodent
272	31		Neotoma	Rodent
652		182	Callipepla	Bird
854	28	47	Pipilo	Bird
977	107	1263	Sylvilagus	Rabbit

Tabular data

- Consistent, tidy data avoids errors.
- Makes data analysis and plotting easier.

variables (columns)



The diagram shows a table with 5 columns and 5 rows. A dashed orange arrow points right above the table, labeled 'variables (columns)'. A dashed orange arrow points down to the left of the table, labeled 'observations (rows)'.

record_id	hindfoot_length	weight	genus	taxa
182	32		Neotoma	Rodent
272	31		Neotoma	Rodent
652		182	Callipepla	Bird
854	28	47	Pipilo	Bird
977	107	1263	Sylvilagus	Rabbit

Tabular data: practical & exercises



- **Live demo** read in data, exploring surveys
- **Practical** work through Chapter 6
([Working with tabular data](#))
- **Exercises** Data structure: surveys,
Subsetting data: parasites



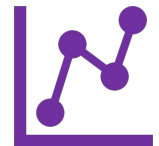
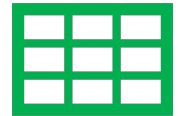
2. Plotting data

Visualising data

Plotting data: requirements

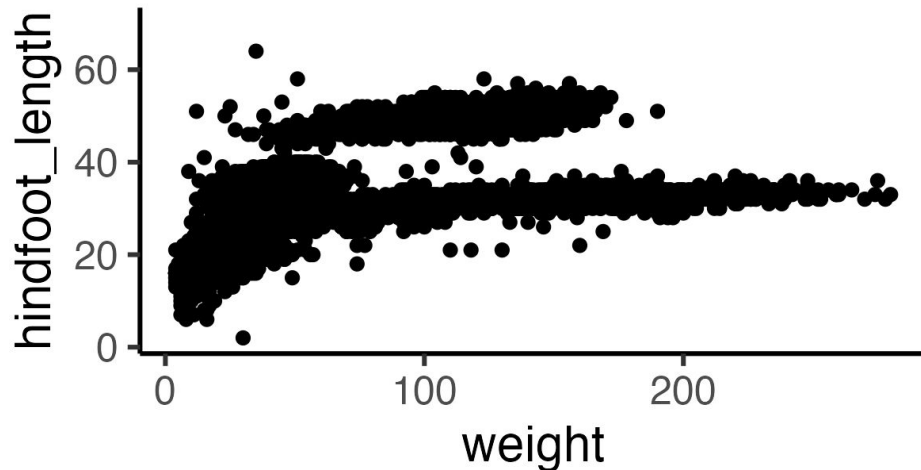
We need three things:

1. Data your data
2. Mapping aesthetics variables used to create the visual (e.g. x/y data, colours)
3. Specify type of plot scatter plot, boxplot etc.



Plotting data: Grammar of graphics

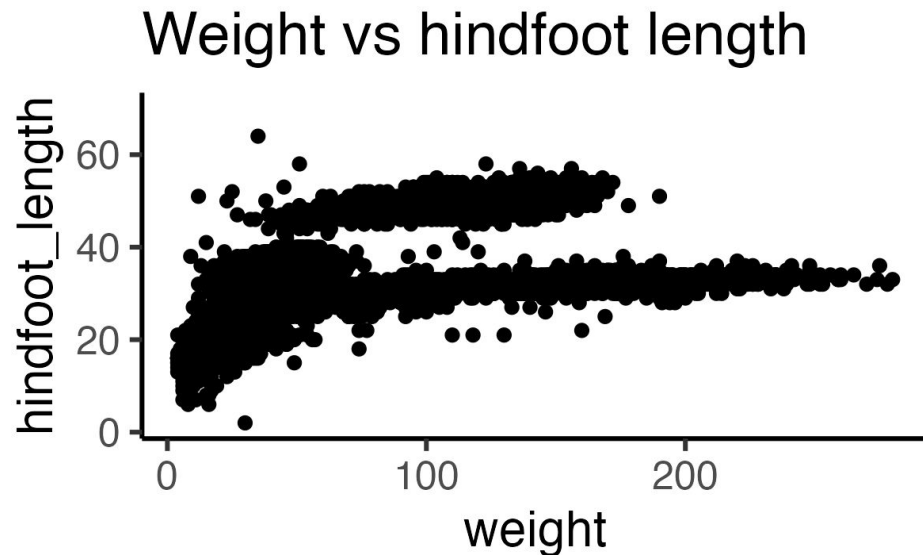
Weight vs hindfoot length



Using `ggplot` in R and
`plotnine` in Python.

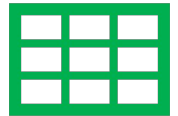
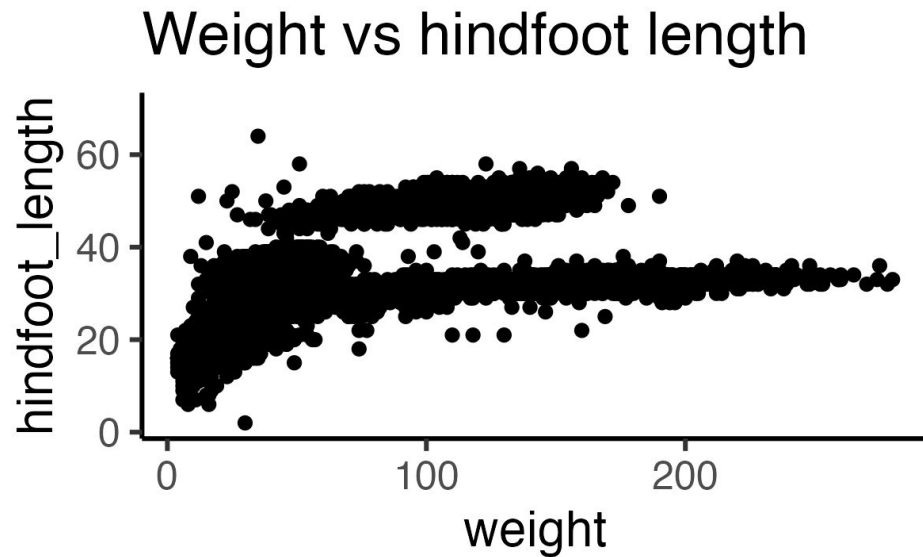
There are other packages in
both languages, some of which
are shown in the course
materials (`seaborn`).

Plotting data: Grammar of graphics



```
ggplot(data = surveys,  
       mapping = aes(x = weight,  
                     y = hindfoot_length)) +  
  geom_point()
```

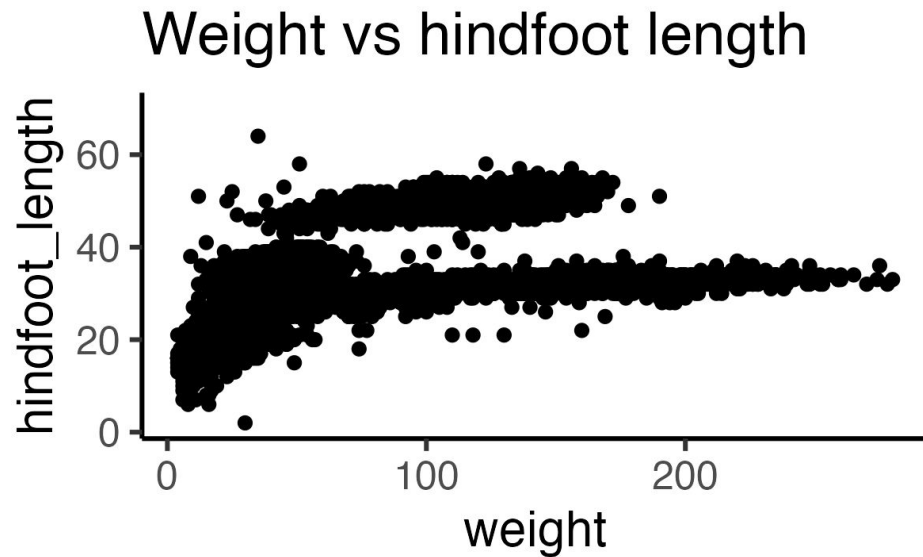
Plotting data: the data



data

```
ggplot(data = surveys,  
        mapping = aes(x = weight,  
                        y = hindfoot_length)) +  
  geom_point()
```

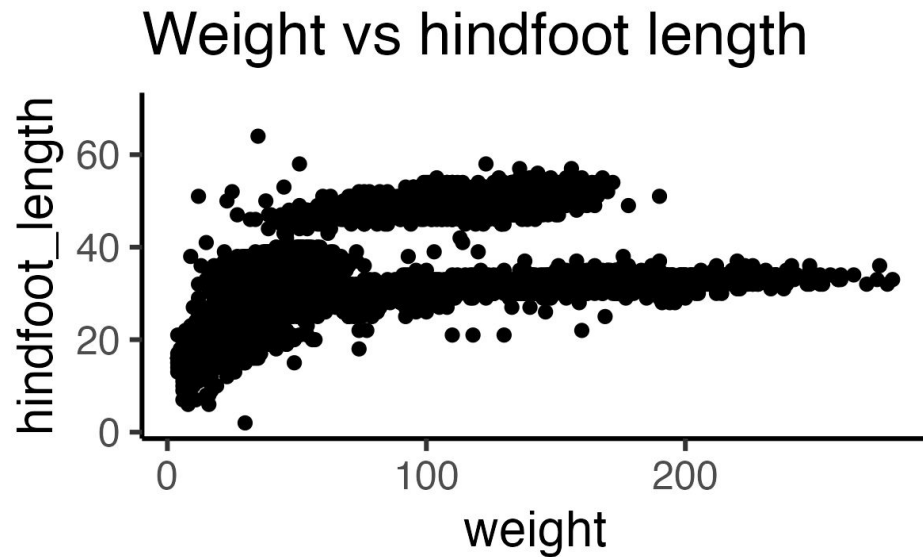
Plotting data: the mapping



variables

```
ggplot(data = surveys,  
       mapping = aes(x = weight,  
                     y = hindfoot_length)) +  
  geom_point()
```

Plotting data: the type of plot



plot type

```
ggplot(data = surveys,  
       mapping = aes(x = weight,  
                     y = hindfoot_length)) +  
  geom_point()
```

Plotting data: practical & exercises



- **Live demo** example
- **Practical** work through Chapter 7
([Plotting data](#))
- **Exercises** Plot colours: parasites,
Facetting plots: parasites



DA2: Summary

- Tabular data contains **variables** in **columns**
- Tabular data has **observations** in **rows**, uniquely identifiable
- We need to consider **missing data** and do common **quality control checks**
- We can select parts of our table with **subsetting**
- Plots require **data**, **mapped variables** and a **plot type**

Coming up next...

DA3: Manipulating data

- Making changes to columns
- Chaining operations
- Manipulating rows
- Performing operations on groups of data



UNIVERSITY OF
CAMBRIDGE

Research Informatics Training

DA2: Plotting tabular data (end)

Cambridge Centre for Research Informatics Training



UNIVERSITY OF
CAMBRIDGE

Research Informatics Training

DA3: Manipulating data

Cambridge Centre for Research Informatics Training

Plan for the session

Session 3: **Manipulating data**

1. Manipulating columns
2. Chaining operations
3. Manipulating rows
4. Grouped operations

1. Manipulating columns

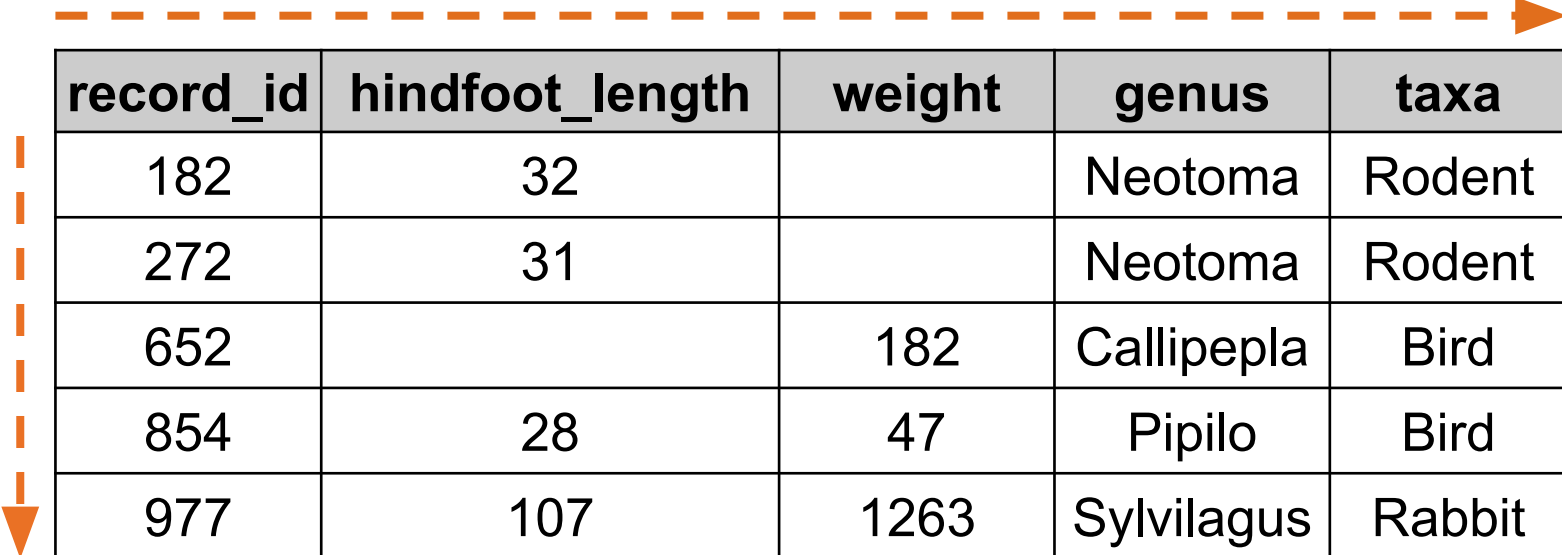
Because default is not enough.

Making changes to tabular data

- Changing columns and/or rows

variables (columns)

observations (rows)



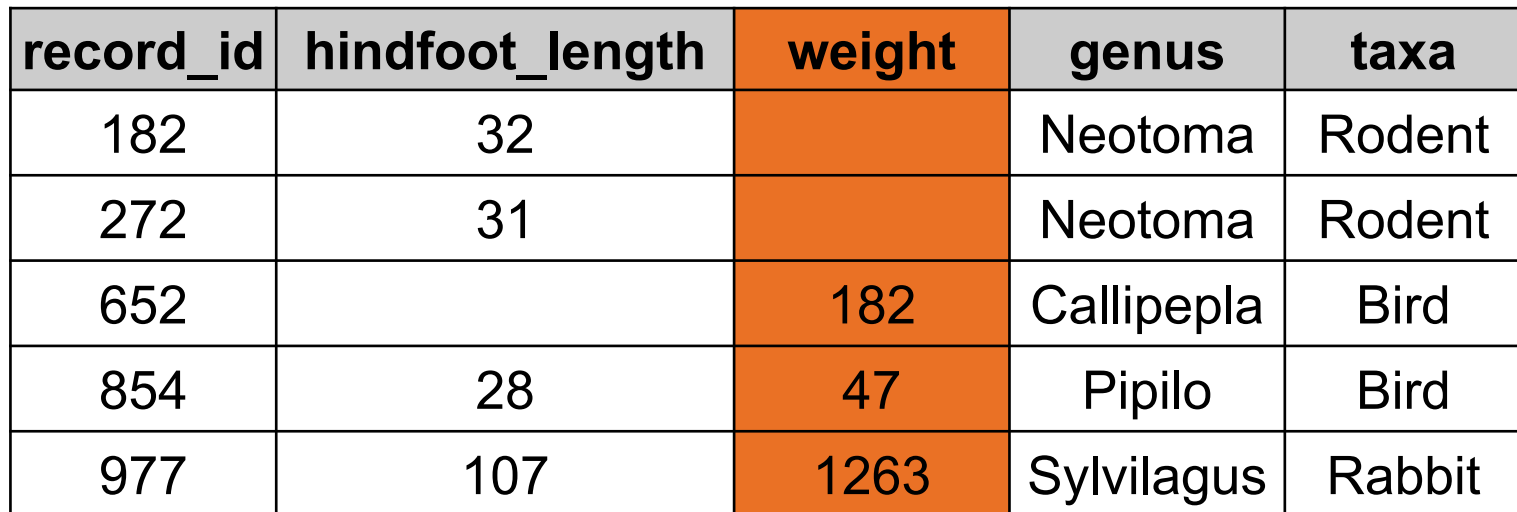
record_id	hindfoot_length	weight	genus	taxa
182	32		Neotoma	Rodent
272	31		Neotoma	Rodent
652		182	Callipepla	Bird
854	28	47	Pipilo	Bird
977	107	1263	Sylvilagus	Rabbit

Making changes to tabular data

- Weight is in grams, what if we want it in kilograms?

variables (columns)

observations (rows)



record_id	hindfoot_length	weight	genus	taxa
182	32		Neotoma	Rodent
272	31		Neotoma	Rodent
652		182	Callipepla	Bird
854	28	47	Pipilo	Bird
977	107	1263	Sylvilagus	Rabbit

Creating new variables

- Weight is in grams, what if we want it in kilograms?

variables (columns) →

↓ observations (rows)

record_id	weight	weight_kg
182		
272		
652	182	0.182
854	47	0.047
977	1263	1.263

For each row:

1. Take weight value
2. Divide by 1,000
3. Store result in new column
4. (skip missing data)

Manipulating columns: practical & exercises



- **Live demo** example
- **Practical** work through Chapter 8
([Manipulating columns](#))
- **Exercises** Selecting columns, creating columns,
creating and plotting columns

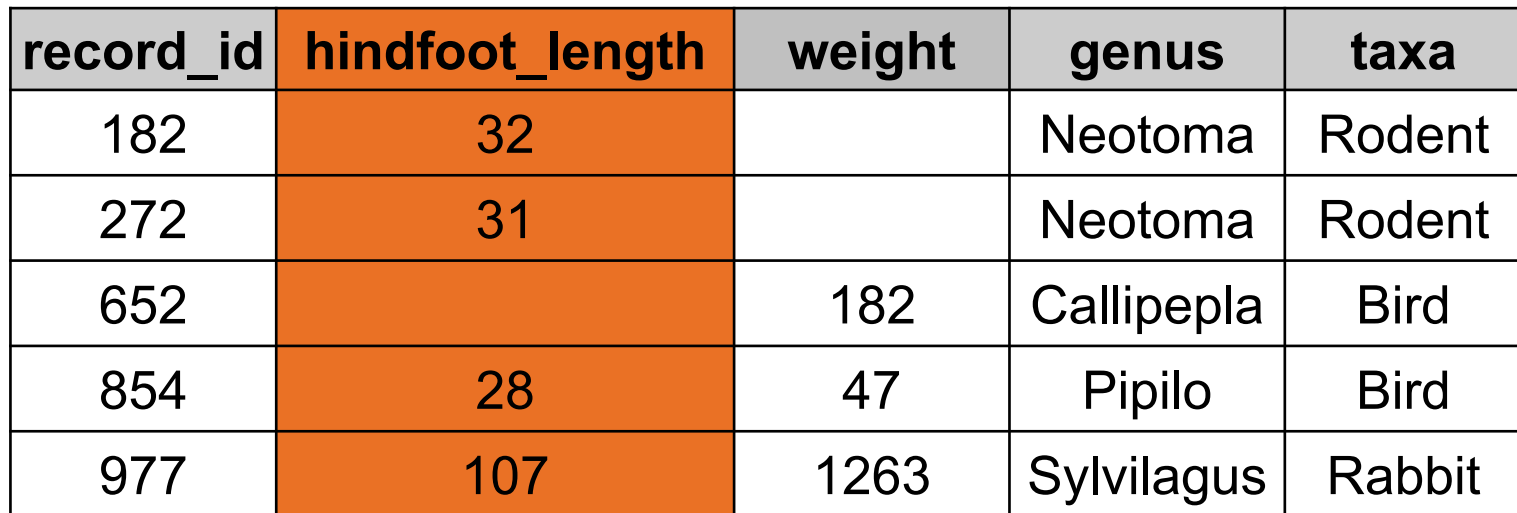


Making changes to tabular data

- Weight is in grams, what if we want it in kilograms?

variables (columns)

observations (rows)



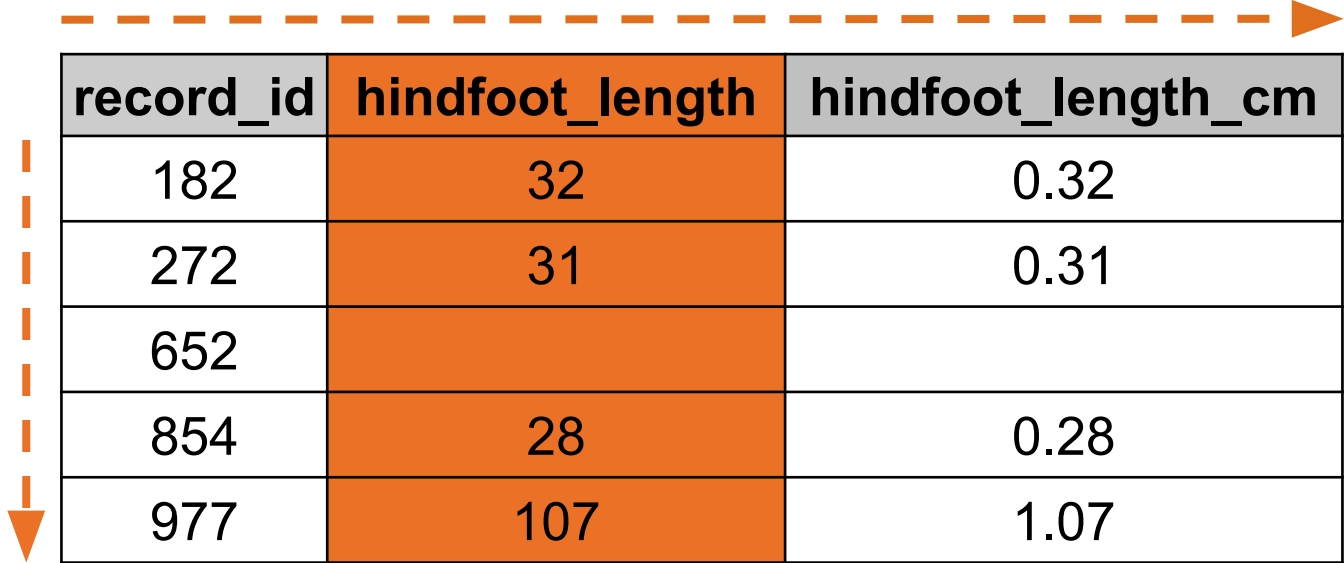
record_id	hindfoot_length	weight	genus	taxa
182	32		Neotoma	Rodent
272	31		Neotoma	Rodent
652		182	Callipepla	Bird
854	28	47	Pipilo	Bird
977	107	1263	Sylvilagus	Rabbit

Making changes to tabular data

- Weight is in grams, what if we want it in kilograms?

variables (columns)

observations (rows)



record_id	hindfoot_length	hindfoot_length_cm
182	32	0.32
272	31	0.31
652		
854	28	0.28
977	107	1.07

2. Chaining operations

Performing operations step-by-step.

Chaining operations

Let's say we wanted to select columns in `surveys`, then create a new column. We could do:

1. Take data, select columns, save in object, e.g.

`surveys_select`

2. Take `surveys_select` and create a new column (e.g. `weight_kg`)

This requires an intermediate step. Better if we could avoid this.

Chaining operations (in R)



Introducing the pipe symbol: `|>` (or `%>%` in tidyverse)

1. Take data, select the columns and send to next function
2. Take the selected data as input and create a new column (e.g. `weight_kg`)

Advantage: step-by-step, without creating intermediate objects!

Chaining operations (in R)



Introducing the pipe symbol: `|>` (or `%>%` in tidyverse)

1. Take data, select the columns and send to next function
2. Take the selected data as input and create a new column (e.g. `weight_kg`)

```
surveys |>
  select(record_id, weight) |>
  mutate(weight_kg = weight / 1000)
```



Chaining operations (in Python)

Introducing the method chaining symbol: `.` (dot)

1. Take data, select the columns and send to next function
2. Take the selected data as input and create a new column (e.g. `weight_kg`)

```
(surveys  
  .loc[:, ["record_id", "weight"]]  
  .assign(weight_kg = lambda df: df["weight"] / 1000))
```

Chaining operations: practical & exercises



- **Live demo** example
- **Practical** work through Chapter 9
([Chaining operations](#))
- **Exercises** Chaining operations



3. Manipulating rows

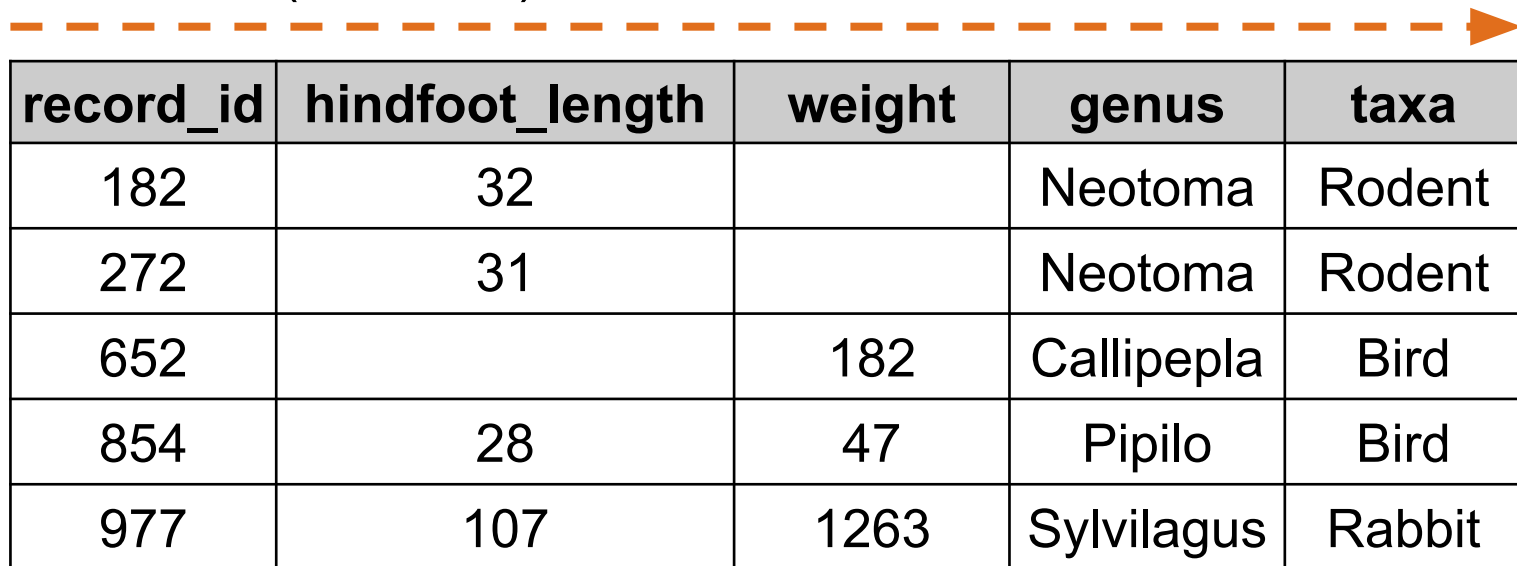
Filtering, ordering and more.

Manipulating rows

- What if we wanted all observations where hindfoot length is greater than 31?

variables (columns)

observations (rows)



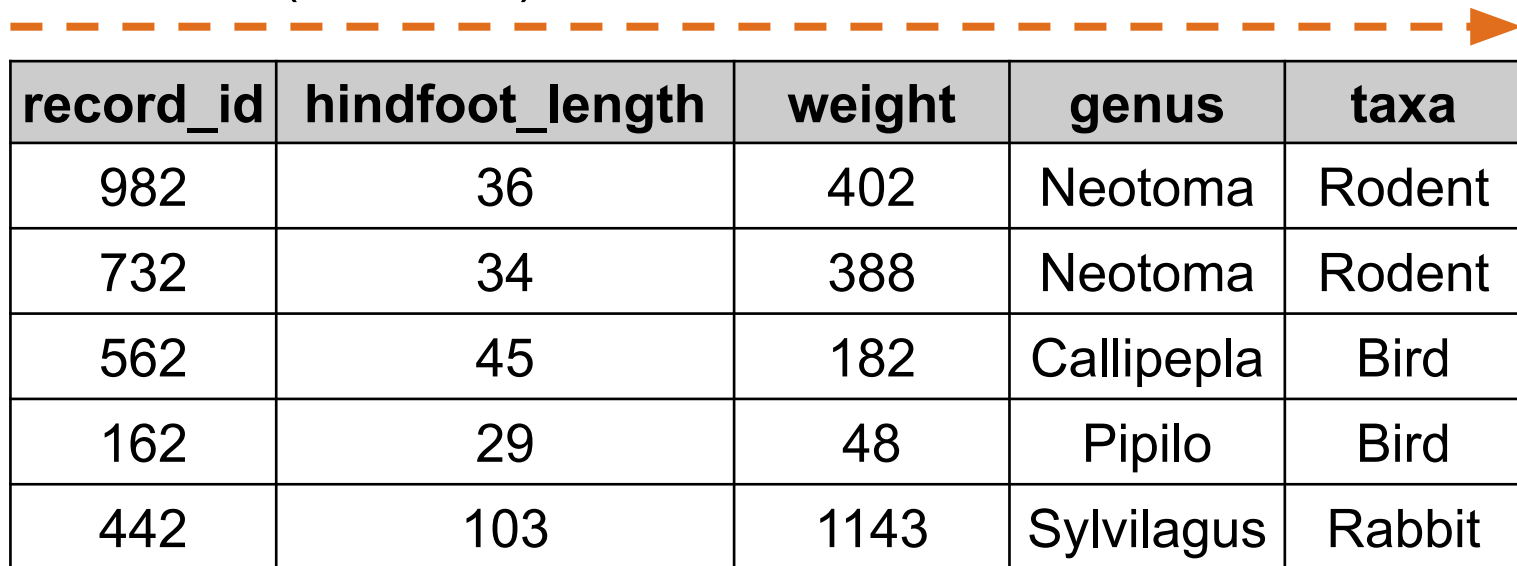
record_id	hindfoot_length	weight	genus	taxa
182	32		Neotoma	Rodent
272	31		Neotoma	Rodent
652		182	Callipepla	Bird
854	28	47	Pipilo	Bird
977	107	1263	Sylvilagus	Rabbit

Making changes to tabular data

- Ordering rows (by weight)

variables (columns)

observations (rows)



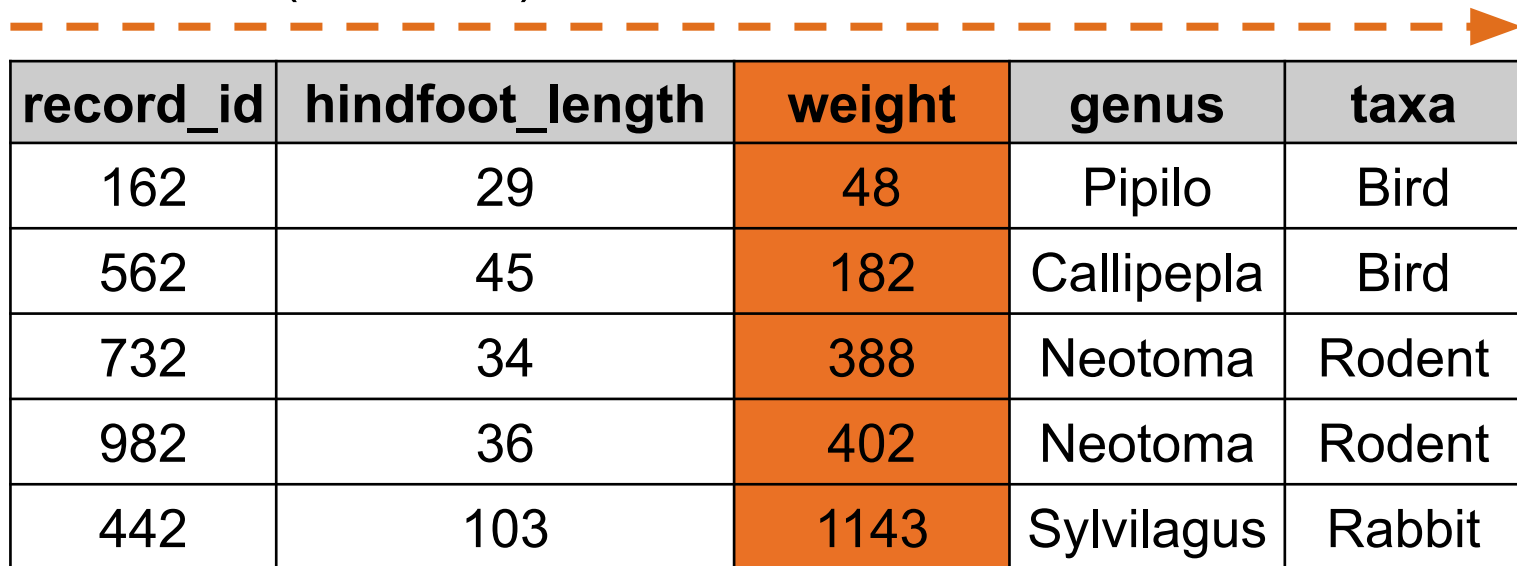
record_id	hindfoot_length	weight	genus	taxa
982	36	402	Neotoma	Rodent
732	34	388	Neotoma	Rodent
562	45	182	Callipepla	Bird
162	29	48	Pipilo	Bird
442	103	1143	Sylvilagus	Rabbit

Making changes to tabular data

- Ordering rows (by weight)

variables (columns)

observations (rows)

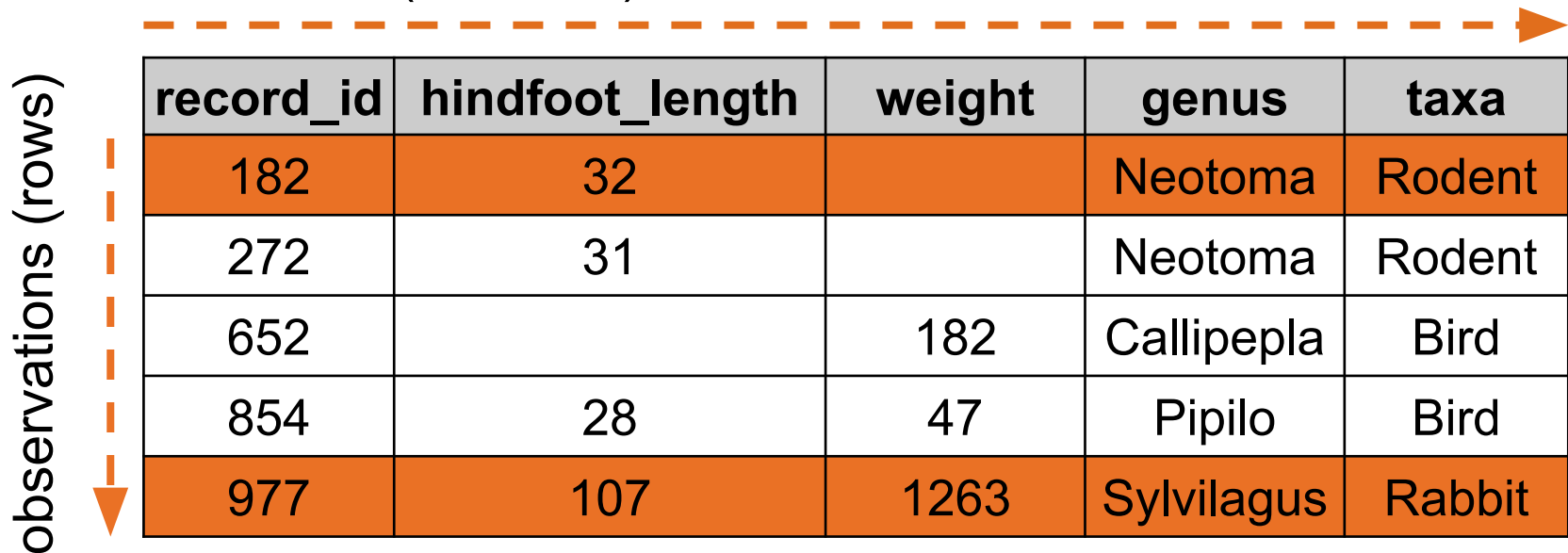


record_id	hindfoot_length	weight	genus	taxa
162	29	48	Pipilo	Bird
562	45	182	Callipepla	Bird
732	34	388	Neotoma	Rodent
982	36	402	Neotoma	Rodent
442	103	1143	Sylvilagus	Rabbit

Making changes to tabular data

- What if we wanted all observations where hindfoot length is greater than 31?

variables (columns)



record_id	hindfoot_length	weight	genus	taxa
182	32		Neotoma	Rodent
272	31		Neotoma	Rodent
652		182	Callipepla	Bird
854	28	47	Pipilo	Bird
977	107	1263	Sylvilagus	Rabbit

Filtering observations

- What if we wanted all observations where hindfoot length is greater than 31?

variables (columns)

observations (rows)

record_id	hindfoot_length	outcome?
182	32	TRUE
272	31	FALSE
652		FALSE
854	28	FALSE
977	107	TRUE

For each row:

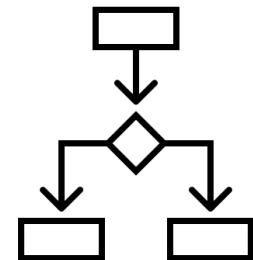
1. Check the value
2. Is it larger than 31?
3. If so, return **TRUE**, otherwise **FALSE**
4. Return all rows where outcome is **TRUE**

Conditional statements

Backbone of **computer logic**:

`if <something> then <something> else <something>`

```
if hindfoot_length > 31  
  then TRUE  
  else FALSE
```



Conditional operators

Operation	R syntax	Python syntax
Equality	<code>==</code>	<code>==</code>
Inequality	<code>!=</code>	<code>!=</code>
Greater than	<code>></code>	<code>></code>
Less than	<code><</code>	<code><</code>
Greater or equal	<code>>=</code>	<code>>=</code>
Less or equal	<code><=</code>	<code><=</code>
Logical AND	<code>&</code>	<code>&, and</code>
Logical OR	<code> </code>	<code> , or</code>
Membership	<code>%in%</code>	<code>in</code>

Manipulating rows: practical & exercises



- **Live demo** example
- **Practical** work through Chapter 10
([Manipulating rows](#))
- **Exercises** Filtering data



4. Grouped operations

Using subgroups in your data.

Grouped operations

Weather data set for different cities:

city	date	temperature	rainfall
London	2024-07-06	24	2
Paris	2024-07-06	22	11
London	2024-08-01	25	2
Paris	2024-08-01	34	0
London	2024-08-06	26	0
...	...	...	...

Calculate average temperature by city.

Grouped operations

split

divide into groups

city	temperature
Paris	22
Paris	34

city	temperature
London	24
London	25
London	26

apply

perform calculations

city	temperature
Paris	28

city	temperature
London	25

combine

aggregate output

city	temperature
Paris	28
London	25

Grouped operations

split

divide into groups

apply

perform calculations

combine

aggregate output

sum

mean

minimum

maximum

transformations

...

Grouped operations: practical & exercises



- **Live demo** example
- **Practical** work through Chapter 11
(Grouped operations)
- **Exercises** Grouped summaries, grouped operations
(infections data set)



DA3: Summary

- We can create **new columns** based on existing ones
- **Filtering** checks conditions for each row - if true, the row is returned
- **Conditional operators** allow us to set the conditions
- We can **chain** operations by going through them sequentially
- **Grouped operations** allow us to apply an operation on groups within a data set

Coming up next...

DA4: Organise & combine

- Reshaping data: from long to wide & back
- Combining data through joins
- Cleaning, styling and arranging plots (bonus)



UNIVERSITY OF
CAMBRIDGE

Research Informatics Training

DA3: Manipulating data (end)

Cambridge Centre for Research Informatics Training



UNIVERSITY OF
CAMBRIDGE

Research Informatics Training

DA4: Organise & combine data

Cambridge Centre for Research Informatics Training

Plan for the session

Session 4: **Organise and combine**

1. Reshaping data
2. Combining data
3. Clean, style & arrange (bonus)

1. Reshaping data

From long to wide & back.

Reshaping data

- What if we wanted to calculate the weight difference year-on-year?
- What if we wanted to plot 2021 against 2022?

record_id	year	weight
199	2021	98
199	2022	83
542	2021	182
542	2022	161
823	2021	203
823	2022	166

Reshaping data

Long format

record_id	year	weight
199	2021	98
199	2022	83
542	2021	182
542	2022	161
823	2021	203
823	2022	166

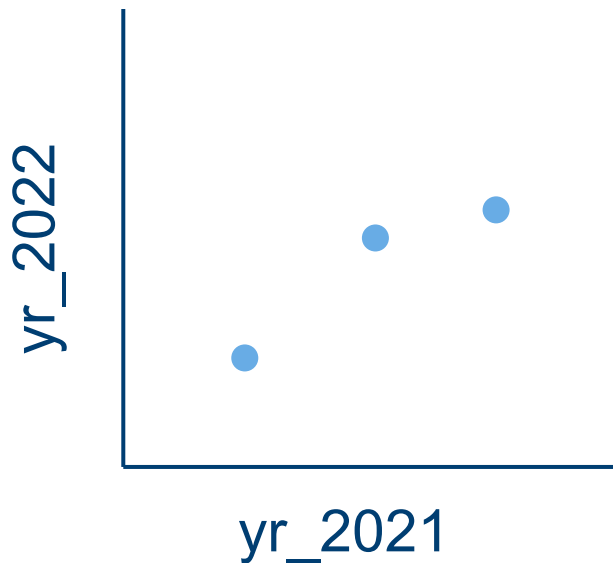
Wide format

record_id	yr_2021	yr_2022
199	98	83
542	182	161
823	203	166

Reshaping data

record_id	yr_2021	yr_2022	difference
199	98	83	15
542	182	161	21
823	203	166	37

Calculate difference



Plot variables

Reshaping data: practical & exercises



- **Live demo** example
- **Practical** work through Chapter 12 ([Reshaping data](#))
- **Exercises** Reshaping data: auxin



2. Combining data

Joining tables together.

Combining data

table a

record_id	plot_id
167	1
192	2
323	3

table b

plot_id	plot_type
1	Spectab enclosure
2	Control
4	Long-term Krat Enclosure

Combining data

table a

record_id	plot_id
167	1
192	2
323	3

+

table b

plot_id	plot_type
1	Spectab enclosure
2	Control
4	Long-term Krat Enclosure

=

Is there a common identifier?

Combining data

table a

record_id	plot_id
167	1
192	2
323	3

table b

plot_id	plot_type
1	Spectab enclosure
2	Control
4	Rodent Enclosure



combined table

record_id	plot_id	plot_type
167	1	Spectab enclosure
192	2	Control
323	3	NA

Left join
(join b to a)

Combining data

table a

record_id	plot_id
167	1
192	2
323	3



table b

plot_id	plot_type
1	Spectab enclosure
2	Control
4	Rodent Enclosure

combined table

plot_id	plot_type	record_id
1	Spectab enclosure	167
2	Control	192
4	Rodent Enclosure	NA

Right join
(join a to b)

Combining data

table a

record_id	plot_id
167	1
192	2
323	3



table b

plot_id	plot_type
1	Spectab enclosure
2	Control
4	Rodent Enclosure

combined table

record_id	plot_id	plot_type
167	1	Spectab enclosure
192	2	Control

Inner join
(retain rows present in both)

Combining data

table a

record_id	plot_id
167	1
192	2
323	3



table b

plot_id	plot_type
1	Spectab enclosure
2	Control
4	Rodent Enclosure

combined table

record_id	plot_id	plot_type
167	1	Spectab enclosure
192	2	Control
323	3	NA
NA	4	Rodent Enclosure

Full join
(retain all rows)

Combining data: practical & exercises



- **Live demo** example
- **Practical** work through Chapter 13
(Combining data)
- **Exercises** Simple joins: surveys



4. Clean, style & arrange

Practical tips & tricks.

Clean, style & arrange (additional materials)

Practical materials: Clean, style & arrange

- Cleaning data (variable names, dealing with numbers, encoding issues, boolean values)
- Cleaning plots (titles, legends, axes labels)
- Changing colours (colour-blind friendly)
- Arranging plots

DA4: Summary

- We can have the same data in a “long” or “wide” format
- Which one you use depends on what you’re doing with the data
- You can also switch between the two if there is a unique identifier column
- We can combine tables if there is an identifier column
- This can be done in different ways (left, right, inner, full)



UNIVERSITY OF
CAMBRIDGE

Research Informatics Training

DA4: Organise & combine data (end)

Cambridge Centre for Research Informatics Training

Final notes

- Course materials & slides will remain online
- Any scripts will be shared via the course site
- Feedback survey accessible via the course site

Related courses: Core statistics



UNIVERSITY OF
CAMBRIDGE

Research Informatics Training

Data analysis in R and Python

Centre for Research Informatics Training